

[https://moodle.joinville.udesc.br/pluginfile.php/428989/mod\\_resource/content/3/PAA\\_01b.pdf](https://moodle.joinville.udesc.br/pluginfile.php/428989/mod_resource/content/3/PAA_01b.pdf)

# Projeto e Análise de Algoritmos

## Programação básica

André Tavares da Silva  
andre.silva@udesc.br

# Análise de Algoritmos

---

Quando utilizamos uma linguagem de programação, é importante saber o funcionamento da mesma para tirar o máximo proveito dela evitando fazer códigos muito lentos.

# Análise de Algoritmos

---

Quando utilizamos uma linguagem de programação, é importante saber o funcionamento da mesma para tirar o máximo proveito dela evitando fazer códigos muito lentos.

Já dizia o grande filósofo: existem duas abordagens de programação: lenta, rápida

# Análise de Algoritmos

---

Quando utilizamos uma linguagem de programação, é importante saber o funcionamento da mesma para tirar o máximo proveito dela evitando fazer códigos muito lentos.

Já dizia o grande filósofo: existem duas abordagens de programação: lenta, rápida e errada.

# Análise de Algoritmos

---

Quando utilizamos uma linguagem de programação, é importante saber o funcionamento da mesma para tirar o máximo proveito dela evitando fazer códigos muito lentos.

Já dizia o grande filósofo: existem duas abordagens de programação: lenta, rápida e errada. Esta última ninguém deveria fazer. ;-)

# Exemplo 1: multiplicação de matrizes em Python

---

```
def multiplicacao_matriz(A, B):  
    if len(A[0]) != len(B):  
        return "Erro: matrizes incompatíveis"  
  
    # Obtém as dimensões das matrizes  
    lins_A = len(A)  
    cols_A = len(A[0])  
    lins_B = len(B)  
    cols_B = len(B[0])  
  
    C = [[0 for _ in range(cols_B)] for _ in range(lins_A)]  
  
    for i in range(lins_A): # Percorre as linhas de A  
        for j in range(cols_B): # Percorre as colunas de B  
            for k in range(cols_A): # Ou num_linhas_B, que é o mesmo  
                C[i][j] += A[i][k] * B[k][j]  
  
    return C
```

# Exemplo 1: multiplicação de matrizes em Python

---

```
# usando numpy
```

```
C = A @ B
```

# Exemplo 1: multiplicação de matrizes em Python

---

Executando...



## Exemplo 2: multiplicação de matrizes em Python

---

Some uma matriz  $10 \times 10$  de zeros com uma matriz (também  $10 \times 10$ ) de números ordenados (1 a 100) e depois multiplique pela matriz identidade ( $10 \times 10$ ). Por fim, some todos os valores da matriz resultante e exiba a resposta ao final.

## Exemplo 2: multiplicação de matrizes em Python

---

Some uma matriz  $10 \times 10$  de zeros com uma matriz (também  $10 \times 10$ ) de números ordenados (1 a 100) e depois multiplique pela matriz identidade ( $10 \times 10$ ). Por fim, some todos os valores da matriz resultante e exiba a resposta ao final.

Executando...

## Exemplo 2: multiplicação de matrizes em Python

---

Some uma matriz  $10 \times 10$  de zeros com uma matriz (também  $10 \times 10$ ) de números ordenados (1 a 100) e depois multiplique pela matriz identidade ( $10 \times 10$ ). Por fim, some todos os valores da matriz resultante e exiba a resposta ao final.

Tem como fazer ainda mais rápido? Soma de Gauss:

<https://www.youtube.com/watch?v=dV7RunfA4q8>

# Desenhando linhas

---

Nem sempre a redução da quantidade de linhas torna o algoritmo mais eficiente. O caso da multiplicação de matrizes é por conta de usar for em Python, que é interpretado, não consegue ser tão eficiente quanto os métodos implementados no numpy (em C).

Vejamos o caso do problema de desenho de linha na tela. Os valores de pixels na tela são valores inteiros e dependem da resolução da tela. Se fizermos uma linha na diagonal, invariavelmente alguns valores da linha cairão entre dois pixels.

Vamos ver dois algoritmos para desenho de linha.

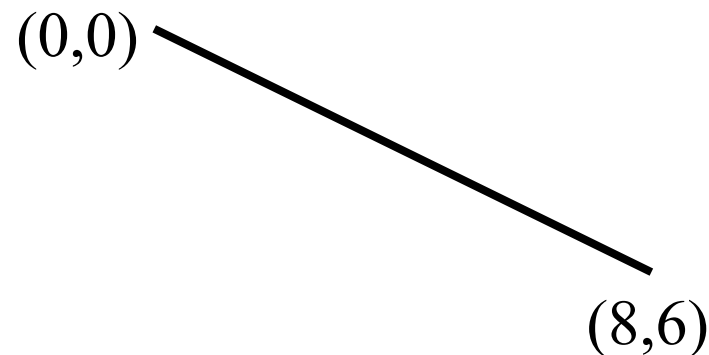
# Desenhando linhas

---

- Método incremental - DDA

Calcular os pontos

```
Dx=xf-xi;  
Dy=yf-yi;  
m=Dy/Dx;  
y=yi;  
for (x=xi;x<=xf;x++) {  
    draw(x,int(floor(y+0.5), color);  
    y+=m;  
}
```



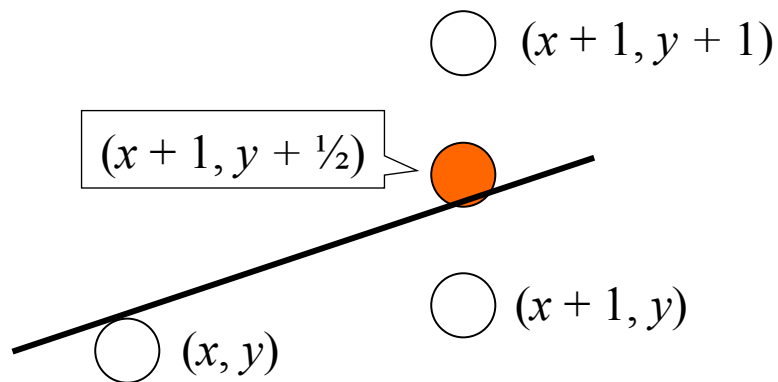
Resposta

(0,0);(1,1);(2,2);(3,2);(4,3);(5,4);(6,5);(7,5);(8,6)

# Desenhando linhas

---

- Algoritmo do Ponto Médio – Bresenham
  - Usa somente operações aritméticas (não usa round ou floor)
- Idéia básica:
  - Em vez de calcular o valor do próximo  $y$  em ponto flutuante, decidir se o próximo pixel vai ter coordenadas  $(x + 1, y)$  ou  $(x + 1, y + 1)$
  - Decisão avalia se a linha passa acima ou abaixo do ponto médio  $(x + 1, y + \frac{1}{2})$



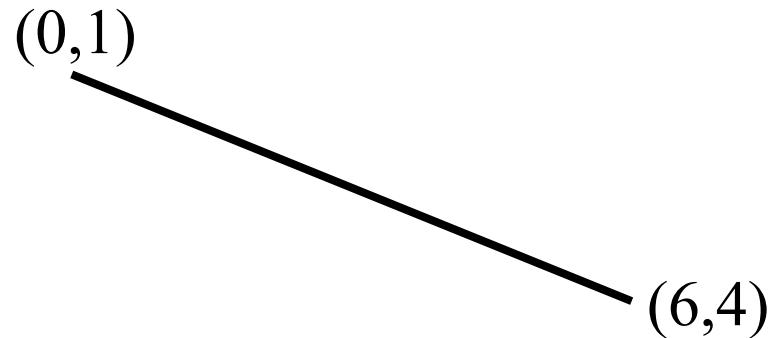
# Desenhando linhas

---

## Algoritmo do Ponto Médio – Bresenham

```
dx = xf - xi;  
dy = yf - yi;  
d = 2*dy - dx;  
y = yi;  
for (x=xi; x <= xf; x++) {  
    draw(x, y,color);  
    if(d > 0) {  
        y++;  
        d -= 2*dx;  
    }  
    d += 2*dy;  
}
```

**Calcular os pontos**



**Resposta**

**(0,1);(1,1);(2,2);(3,2);(4,3);(5,3);(6,4)**

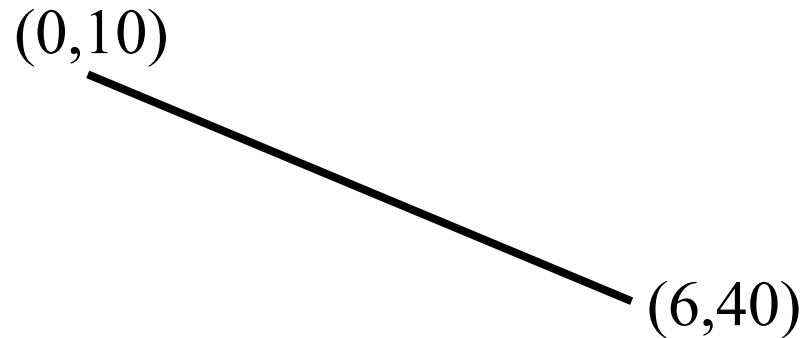
# Desenhando linhas

---

## Algoritmo do Ponto Médio – Bresenham

```
dx = xf - xi;  
dy = yf - yi;  
d = 2*dy - dx;  
y = yi;  
for (x=xi; x <= xf; x++) {  
    draw(x, y,color);  
    if(d > 0) {  
        y++;  
        d -= 2*dx;  
    }  
    d += 2*dy;  
}
```

**Calcular os pontos**



**E se  $dx < dy$ ?**

**Resposta**

**(0,10);(1,11);(2,12);(3,13);(4,14);(5,15);(6,16)**



# Desenhando linhas

## Algoritmo do Ponto Médio – Bresenham

```
dx = xf - xi;  
dy = yf - yi;  
d = 2*dy - dx;  
y = yi;  
for (x=xi; x <= xf; x++) {  
    draw(x, y,color);  
    if(d > 0) {  
        y++;  
        d -= 2*dx;  
    }  
    d += 2*dy;  
}
```

**Calcular os pontos**

(0,1)

(6,4)

**E se  $dx < dy$ ?**

**E se  $xi > xf$ ?**

**E se  $dx < dy$  e  $yi > yf$ ?**

## Exemplo 2: programando em C++

---

Preciso ler um arquivo numérico e armazenar em um vetor na memória. Qual estrutura de dados utilizar da STL?

## Exemplo 2: programando em C++

---

Preciso ler um arquivo numérico e armazenar em um vetor na memória. Qual estrutura de dados utilizar da STL?

`std::vector` geralmente oferece o melhor desempenho geral devido à sua localidade de dados e acesso aleatório rápido.

`std::list` é útil para inserções/remoções frequentes no meio da lista.

`std::set` é otimizado para a busca eficiente de elementos únicos e ordenados.

## Exemplo 2: programando em C++

---

Preciso ler um arquivo numérico e armazenar em um vetor na memória. Qual estrutura de dados utilizar da STL?

| Container                | Melhor para                                           | Pior Para                                         |
|--------------------------|-------------------------------------------------------|---------------------------------------------------|
| <code>std::vector</code> | Acesso aleatório, iteração, inserção/remoção no final | Inserção/remoção no meio                          |
| <code>std::list</code>   | Inserção/remoção no meio ou extremos (com iterador)   | Acesso aleatório, iteração sequencial (sem cache) |
| <code>std::set</code>    | Busca, garantia de unicidade, ordenação automática    | Acesso por índice, armazenamento de duplicatas    |

## Exemplo3: Jogos

---

Precisamos agora ler um ambiente formado por milhares de polígonos e precisamos exibí-los em uma alta taxa de quadros (mais de 60 FPS, por exemplo).

Para cada um dos polígonos, precisamos saber se precisamos desenhar. Ou seja, se está dentro do volume de visualização e se é visível pela câmera.

## Exemplo3: Jogos

---

Precisamos agora ler um ambiente formado por milhares de polígonos e precisamos exibí-los em uma alta taxa de quadros (mais de 60 FPS, por exemplo).

Para cada um dos polígonos, precisamos saber se precisamos desenhar. Ou seja, se está dentro do volume de visualização e se é visível pela câmera. Somente os que precisamos processar iremos calcular a cor (iluminação):

$$I_{\lambda} = I_{a\lambda} k_a O_{\lambda} + \sum_{i=1}^m f_{att_i} I_{p\lambda_i} \left[ k_d O_{d\lambda} (N \cdot L_i) + k_s O_{s\lambda} (R_i \cdot V)^n \right]$$

## Exemplo3: Jogos

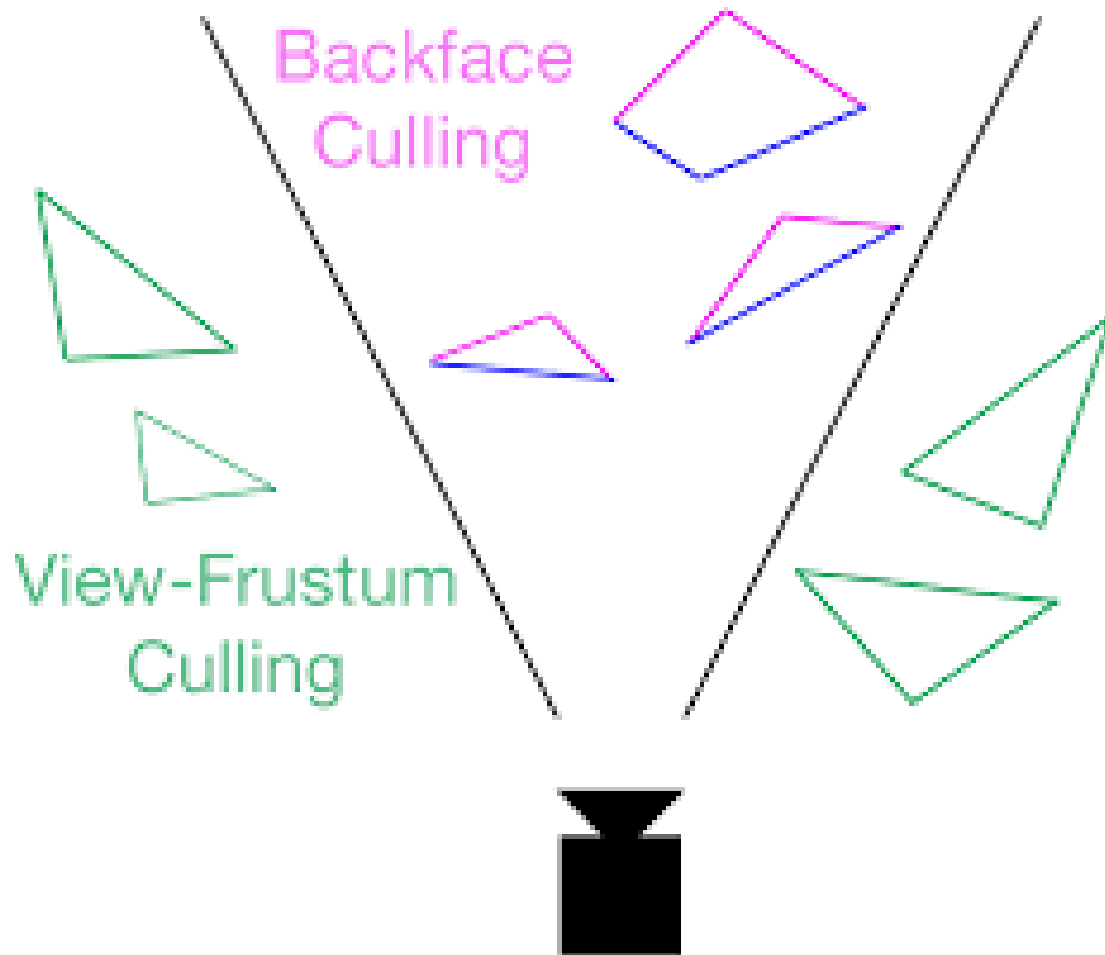
---

Precisamos agora ler um ambiente formado por milhares de polígonos e precisamos exibí-los em uma alta taxa de quadros (mais de 60 FPS, por exemplo).

Para cada um dos polígonos, precisamos saber se precisamos desenhar. Ou seja, se está dentro do volume de visualização e se é visível pela câmera.

# O quê precisamos desenhar?

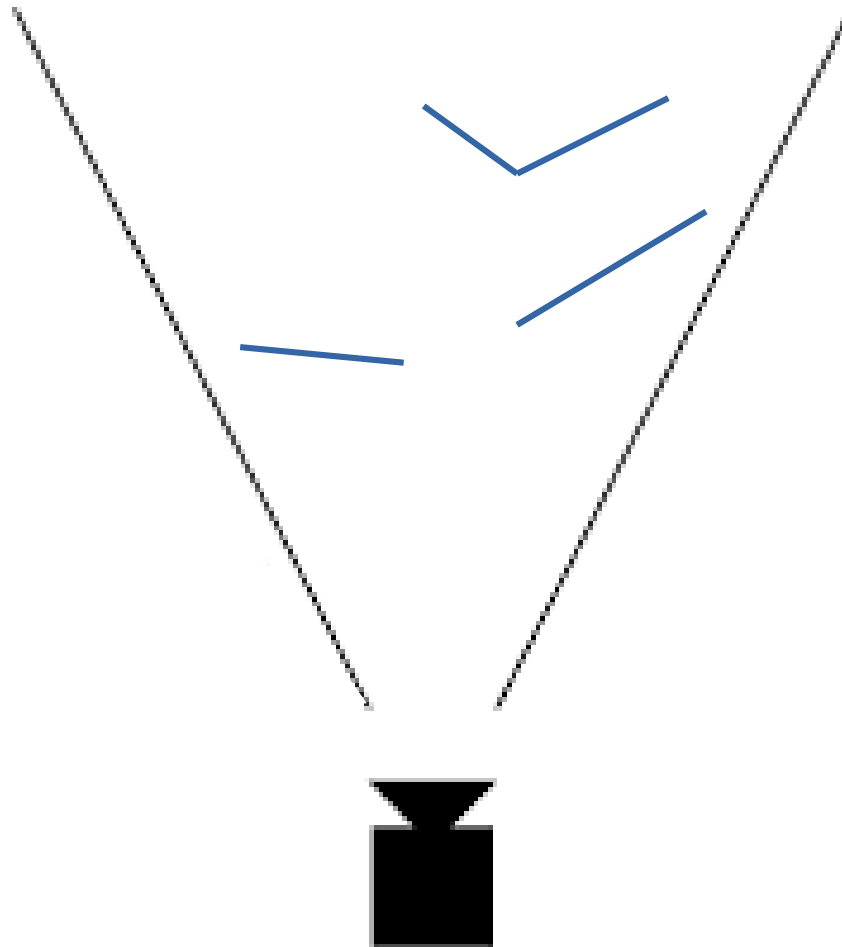
---





# O quê precisamos desenhar?

---



# Jogos – View-Frustum

(se está dentro do volume de visualização)

---

- Bits:
  - Bit 0: ponto acima do volume
  - Bit 1: ponto abaixo do volume
  - Bit 2: ponto direita do volume
  - Bit 3: ponto esquerda do volume
  - Bit 4: ponto atrás do volume
  - Bit 5: ponto na frente do volume

# Jogos – View-Frustum

(se está dentro do volume de visualização)

---

- Bits:
  - Bit 0: ponto acima do volume
  - Bit 1: ponto abaixo do volume
  - Bit 2: ponto direita do volume
  - Bit 3: ponto esquerda do volume
  - Bit 4: ponto atrás do volume
  - Bit 5: ponto na frente do volume
- Todos os bits em zero: trivialmente aceitos.
- AND entre os pontos diferente de zero: trivialmente recusados.
- AND entre os pontos igual a zero: recortar.

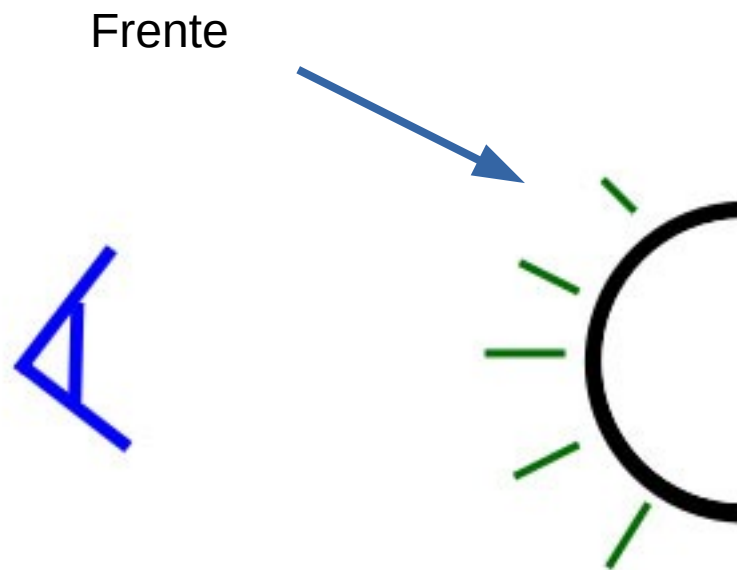
# Jogos – backface culling

---



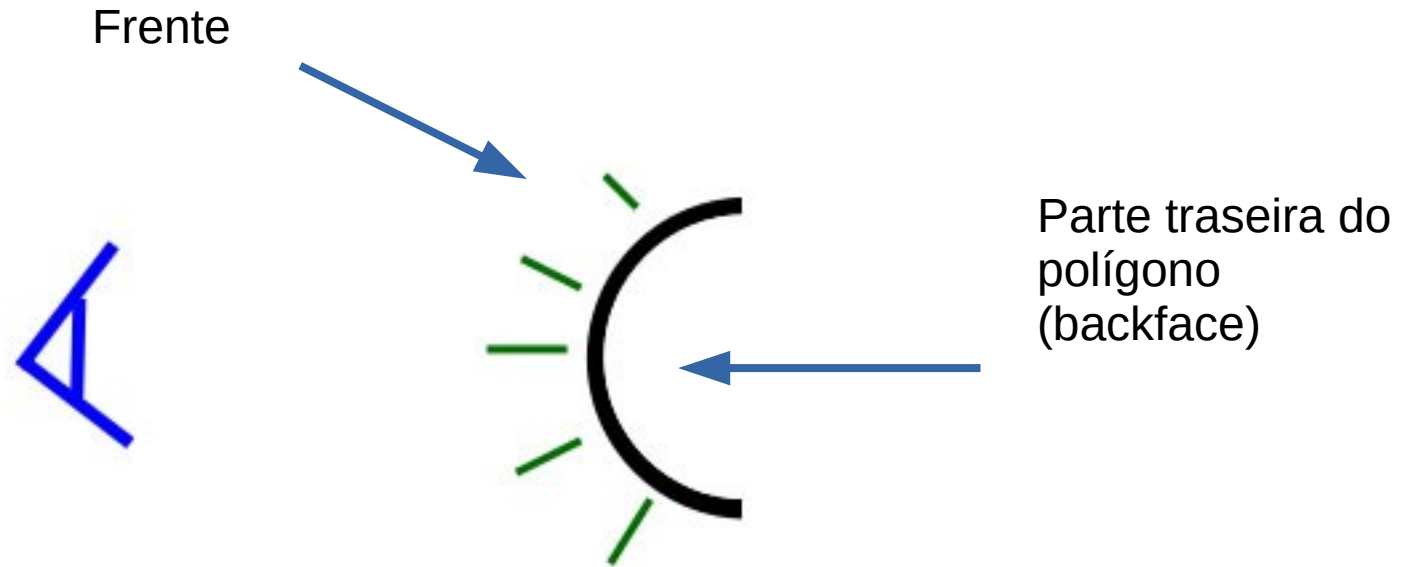
# Jogos – backface culling

---



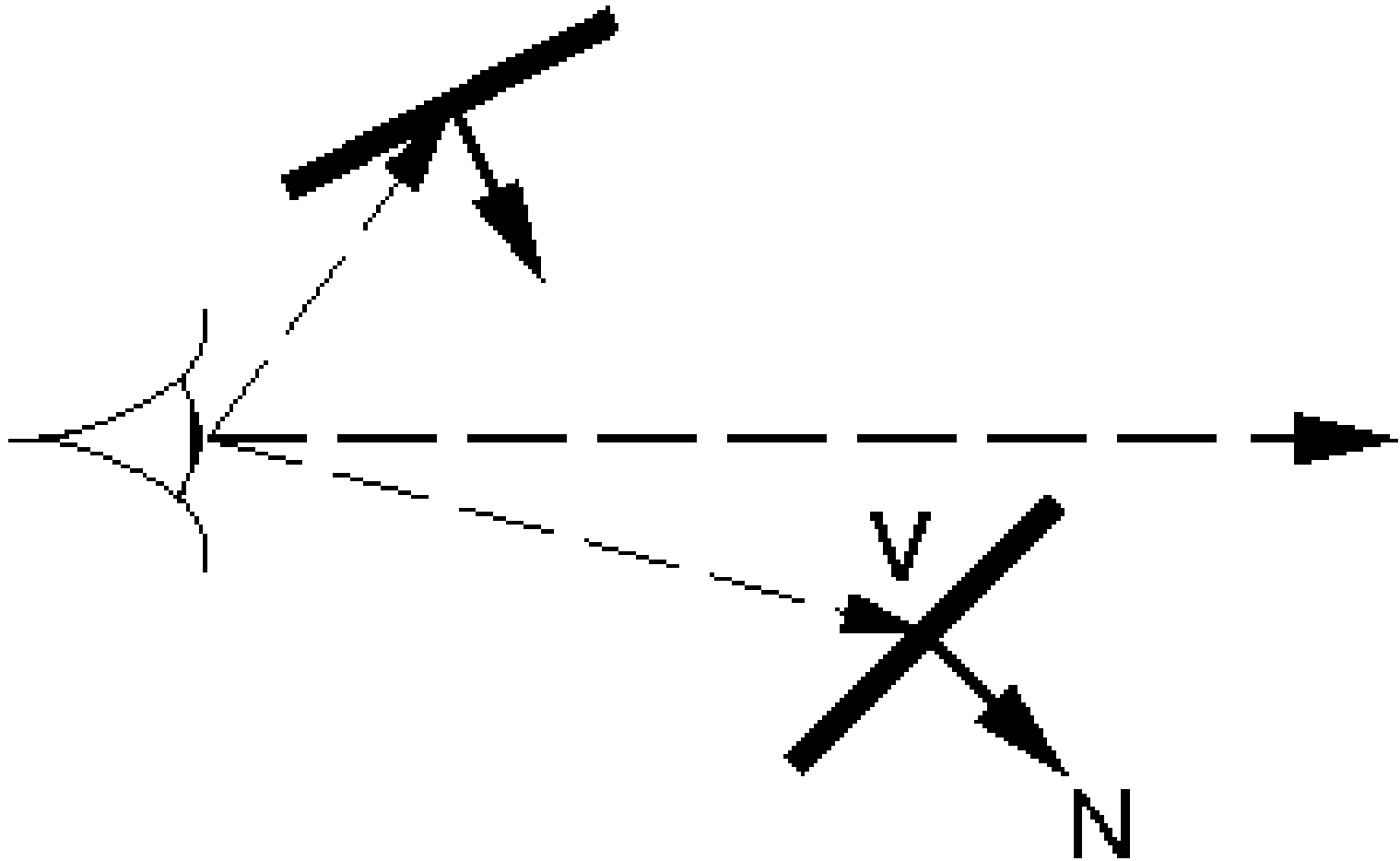
# Jogos – backface culling

---



# Jogos – backface culling

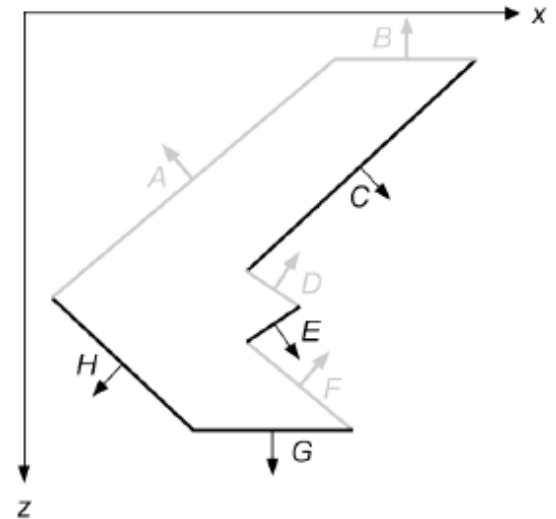
---



# Jogos – backface culling

## *Backface Culling*

- Se  $N.V > 0$ , para objetos sólidos isso significa que essa face nunca será vista pelo observador
- $V$  é um vetor que parte do olho para o objeto

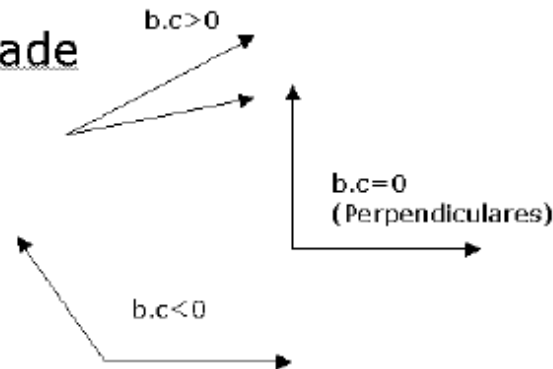


### Sinal de $b.c$ e Perpendicularidade

$$\cos \theta > 0 \text{ se } |\theta| < 90^\circ \rightarrow b.c > 0$$

$$\cos \theta = 0 \text{ se } |\theta| = 90^\circ \rightarrow b.c = 0$$

$$\cos \theta < 0 \text{ se } |\theta| > 90^\circ \rightarrow b.c < 0$$





# Jogos – normalizando um vetor

---

Lembrando que o produto escalar somente será o cosseno do ângulo se os vetores forem normalizados (módulo = 1).

Normalizando um vetor:

$$\hat{u} = \frac{u}{\|u\|} = \left( \frac{u_1}{\|u\|}, \frac{u_2}{\|u\|}, \dots, \frac{u_n}{\|u\|} \right)$$

$$\|u\| = \sqrt{\frac{u_1}{\sqrt{u_1^2 + u_2^2 + \dots + u_n^2}} + \frac{u_2}{\sqrt{u_1^2 + u_2^2 + \dots + u_n^2}} + \dots + \frac{u_n}{\sqrt{u_1^2 + u_2^2 + \dots + u_n^2}}}$$

# Jogos – calculando a raiz quadrada

---

**Método de Newton/Heron (Método Babilônico):** Este é um dos métodos mais antigos e comuns. Começa com uma aproximação inicial e a refina iterativamente usando:  $\text{nova\_aproximacao} = 0.5 * (\text{aprox\_anterior} + (\text{numero} / \text{aprox\_anterior}))$ . Esse processo é repetido até que a diferença entre as aproximações seja muito pequena, garantindo alta precisão.

**Aproximação por Hardware (Instruções de CPU):** Em muitos sistemas modernos, a função `sqrt()` não é implementada puramente em software, mas sim por meio de instruções nativas do processador (como as instruções SSE/AVX em CPUs x86/x64). Isso é extremamente rápido, pois o cálculo é feito diretamente pelo hardware especializado.

**Manipulação de Ponto Flutuante (IEEE 754):** Algumas implementações usam "mágica" em nível de bits, tirando proveito da representação interna de números de ponto flutuante (padrão IEEE 754). Uma famosa técnica (embora mais comum para a raiz quadrada inversa rápida, `rsqrt`) envolve uma manipulação bit a bit para obter uma excelente aproximação inicial, que é então refinada com uma ou duas iterações do método de Newton.

# Jogos – calculando a raiz quadrada

---

Fast Inverse Square Root — A Quake III Algorithm:  
[https://www.youtube.com/watch?v=p8u\\_k2LIZyo](https://www.youtube.com/watch?v=p8u_k2LIZyo)

Precisamos agora ler um ambiente formado por **milhares de polígonos** e precisamos exibí-los **em uma alta taxa de quadros** (mais de 60 FPS, por exemplo).

Para cada um dos polígonos, precisamos saber se precisamos desenhar. **Ou seja, se está dentro do volume de visualização e se é visível pela câmera.** Somente os que precisamos processar iremos calcular a cor (iluminação):

$$I_{\lambda} = I_{a\lambda} k_a O_{\lambda} + \sum_{i=1}^m f_{att_i} I_{p\lambda_i} \left[ k_d O_{d\lambda} (N \cdot L_i) + k_s O_{s\lambda} (R_i \cdot V)^n \right]$$

Precisamos agora ler um ambiente formado por milhares de polígonos e precisamos exibí-los em uma alta taxa de quadros (mais de 60 FPS, por exemplo).

Mas não basta exibir o ambiente. Precisamos popular ele com NPCs. E esses NPCs não podem atravessar por dentro dos demais. Precisamos detectar e evitar colisão entre os NPCs.

# Jogos – detecção de colisão

Mas não basta exibir o ambiente. Precisamos popular ele com NPCs. E esses NPCs não podem atravessar por dentro dos demais. Precisamos detectar e evitar colisão entre os NPCs.



[https://www.youtube.com/watch?v=0k7\\_TXFLyIU&t=2m](https://www.youtube.com/watch?v=0k7_TXFLyIU&t=2m)

# Jogos – detecção de colisão

---

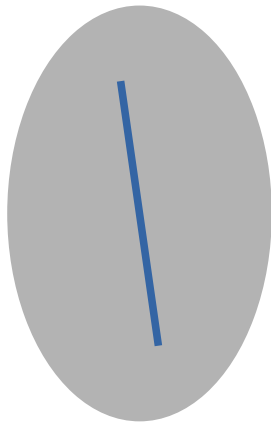
Em vez de testarmos a colisão entre cada um dos polígonos, podemos verificar se existe alguma intersecção entre dois polígonos convexos.

A envoltória convexa (*convex hull*, também chamada de invólucro convexo ou fecho convexo) de um conjunto  $S \in \mathbb{R}^m$  é a intersecção de todos conjuntos convexos que contém  $S$ . Ou seja, é o menor conjunto convexo que contém  $S$ .

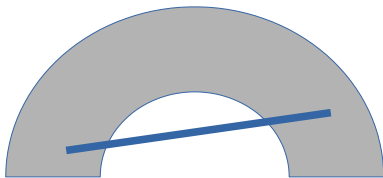
Tal definição pode ser vista como "exterior", pois envolve conjuntos que contém  $S$ . Uma caracterização "interior" é dada por: A envoltória convexa de  $S \in \mathbb{R}^m$  é o conjunto de todas combinações convexas de coleções finitas de pontos de  $S$ .

# Jogos – convex hull

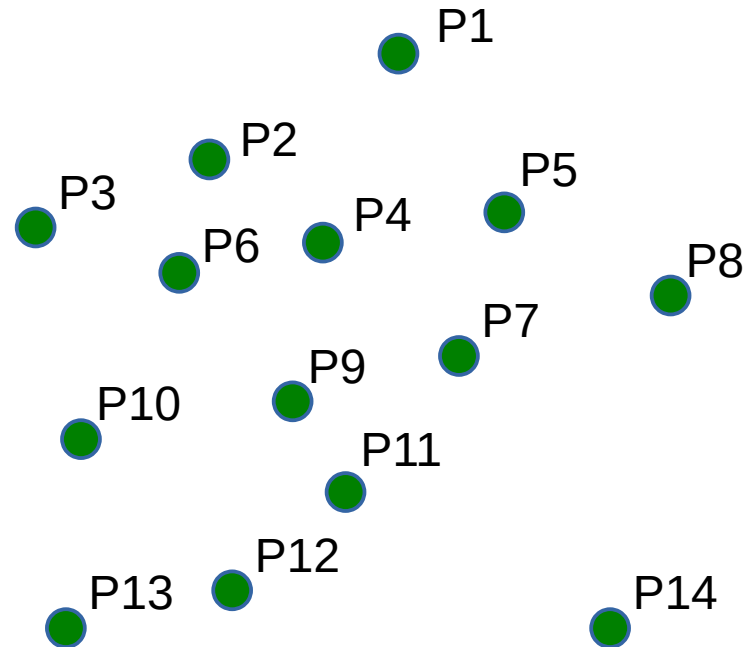
---



Convexo



Não convexo

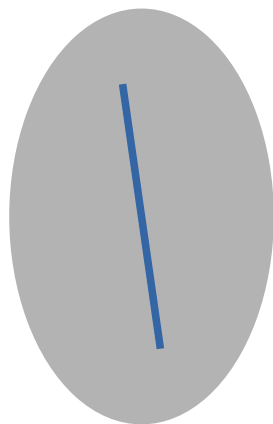




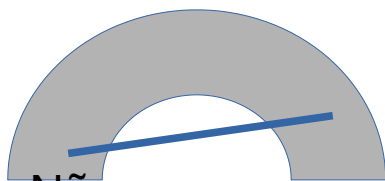
# Convex Hull

(envoltória convexa / fecho convexo)

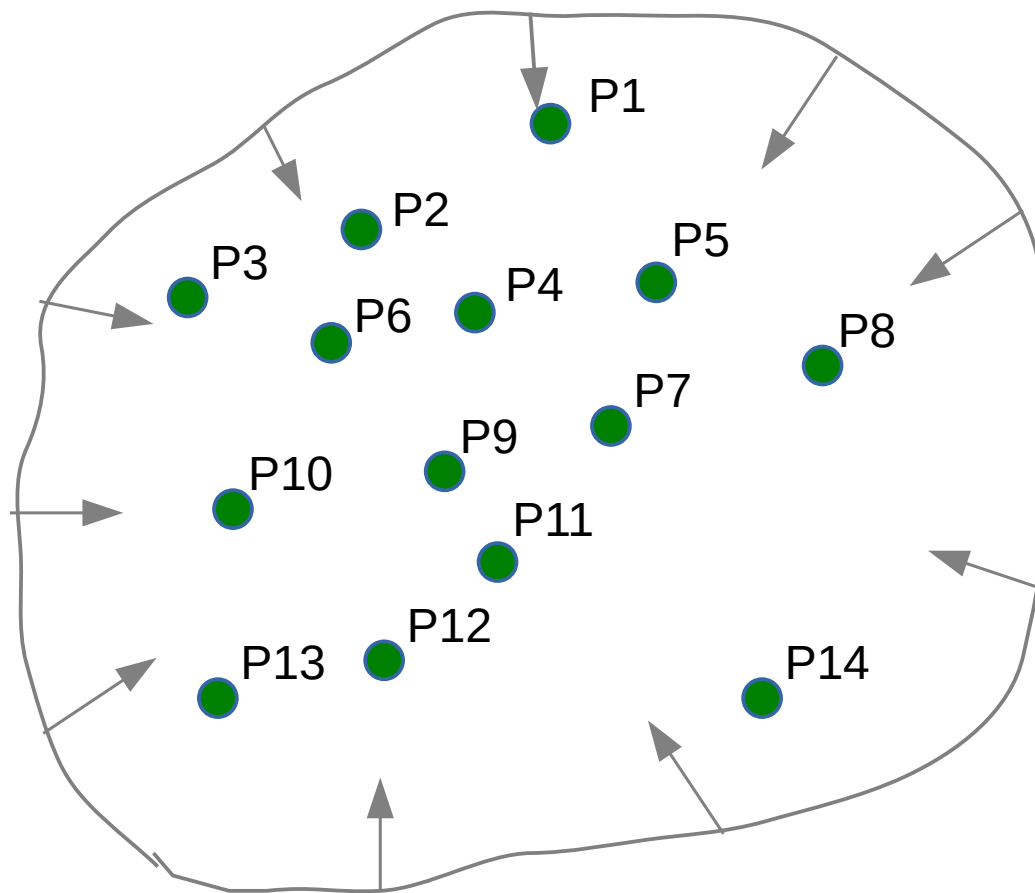
---



Convexo



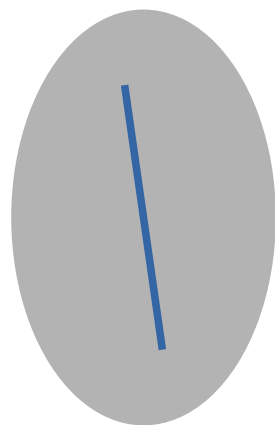
Não convexo



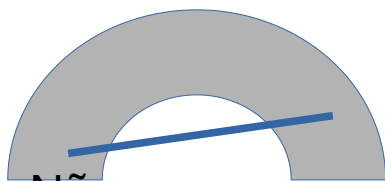
# Convex Hull

(envoltória convexa / fecho convexo)

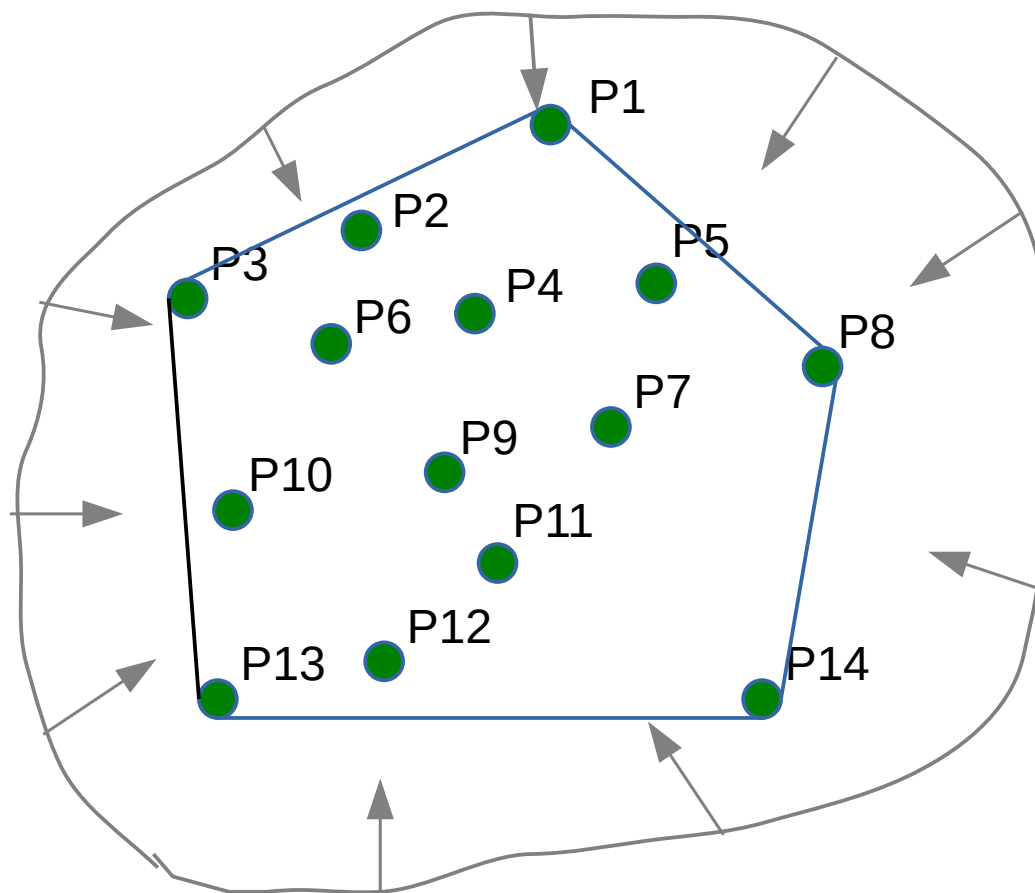
---



Convexo



Não convexo



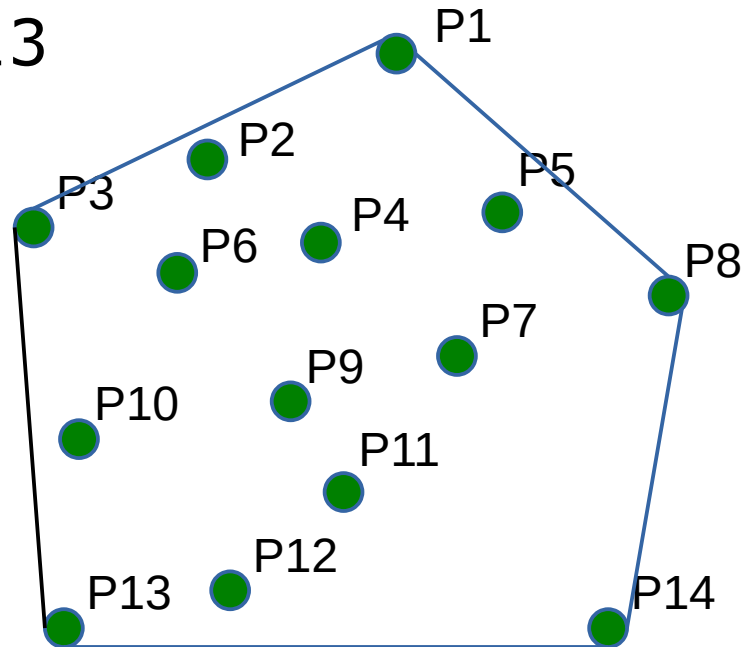
# Convex Hull

(envoltória convexa / fecho convexo)

---

Entrada: P1, P2, P3,...,P14

Saída: P3, P1,P8,P14,P13



# Convex Hull

(envoltória convexa / fecho convexo)

---

*Como saber se um ponto está a direita de um segmento de reta?*

- Ao ter uma linha formada por  $P_0(x_0, y_0)$  até  $P_1(x_1, y_1)$ , um ponto  $P(x, y)$  e a expressão

$$(y - y_0)(x_1 - x_0) - (x - x_0)(y_1 - y_0),$$

podemos dizer que se o valor da expressão for **menor** que 0 então P está à **direita** do segmento de linha, se **maior** que 0 à **esquerda** e se **igual** a 0 então o ponto está **sobre o segmento** de linha.

# Convex Hull

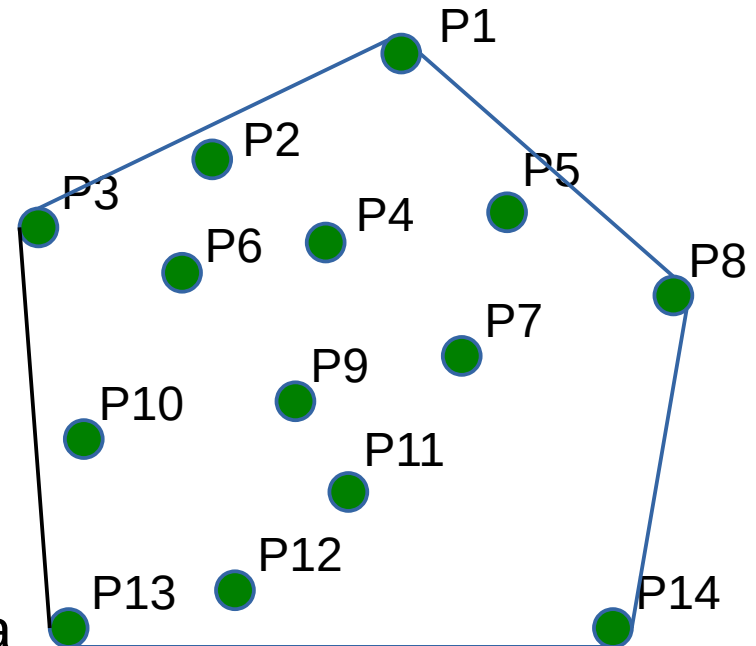
(envoltória convexa / fecho convexo)

---

Entrada: Conjunto de pontos  $P$  no plano

Saída: Uma lista de vértices em ordem

1.  $E = \emptyset$
2. FOR todos pares  $(p,q) \ P \times P \ (p \neq q)$
3. DO valido = true
4. FOR todos pontos  $r \ P (r \neq p \text{ and } r \neq q)$
5. IF  $r$  está a direita da reta  $pq$
6. THEN valido = false
7. IF valido THEN adicione  $pq$  para  $E$
8. Para o conjunto  $E$  de arestas, construa uma lista de vértices classificados em ordem cronológica



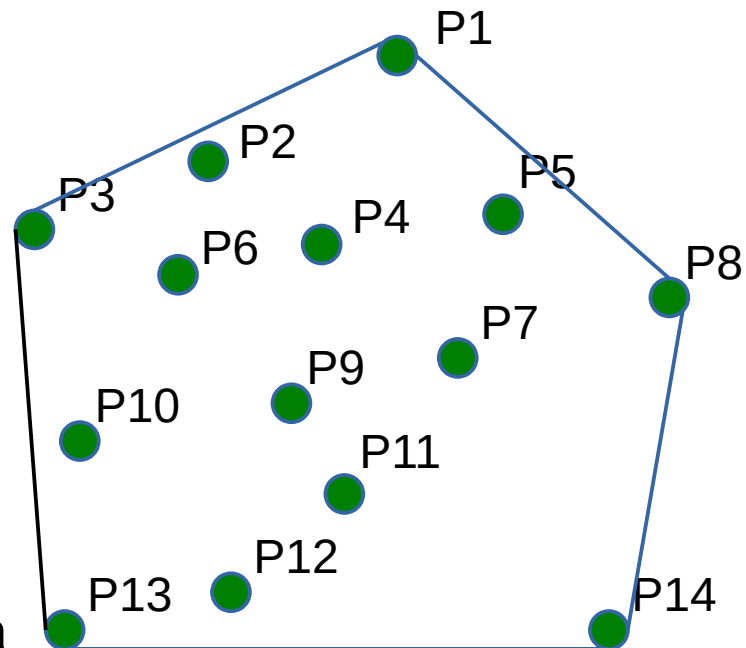
# Convex Hull

(envoltória convexa / fecho convexo)

Entrada: Conjunto de pontos  $P$  no plano

Saída: Uma lista de vértices em ordem

1.  $E = \emptyset$
2. FOR todos pares  $(p,q) P \times P$  ( $p \neq q$ )
3. DO valido = true
4. FOR todos pontos  $r P$  ( $r \neq p$  and  $r \neq q$ )
5. IF  $r$  está a direita da reta  $pq$
6. THEN valido = false
7. IF valido THEN adicione  $pq$  para  $E$
8. Para o conjunto  $E$  de arestas, construa uma lista de vértices classificados em ordem cronológica



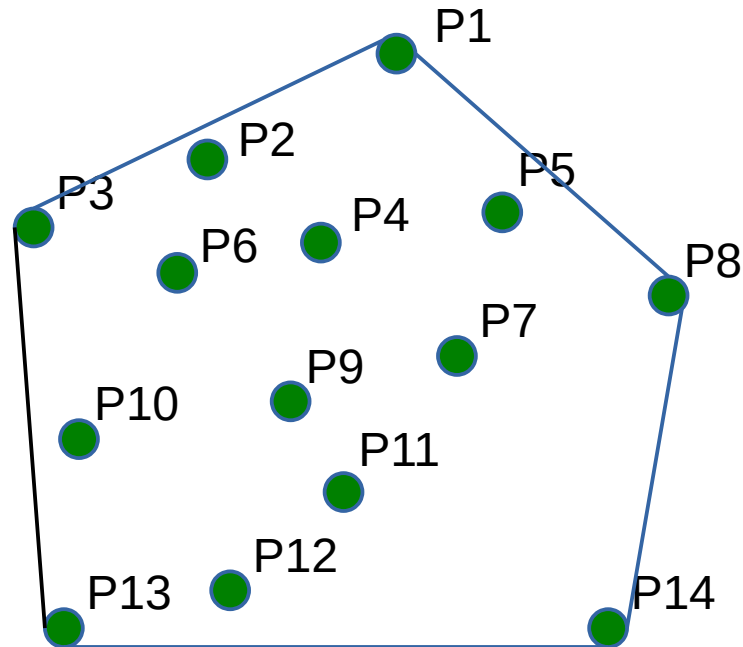
Complexidade:  $n * (n-1) * (n-2) = n^3 - 3n^2 + 2n = \mathbf{n^3}$

# Convex Hull

(envoltória convexa / fecho convexo)

---

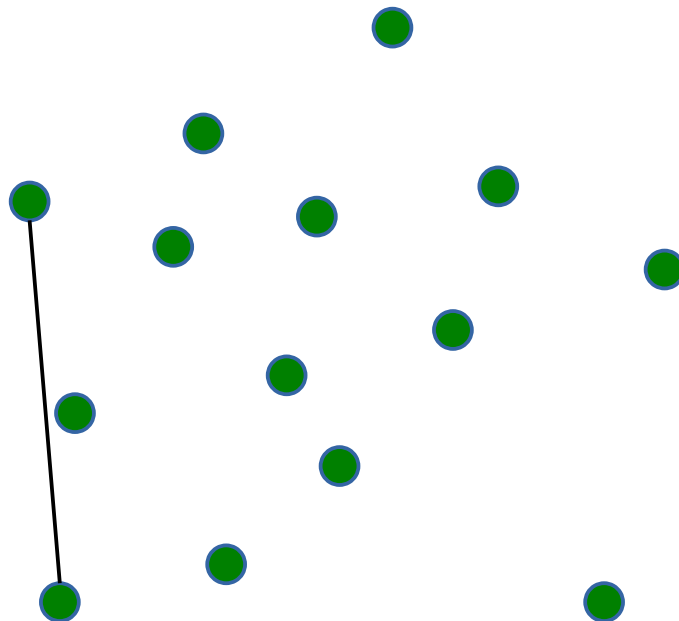
E se os pontos estiverem ordenados?  
E como ordenar?



# Convex Hull

(envoltória convexa / fecho convexo)

---

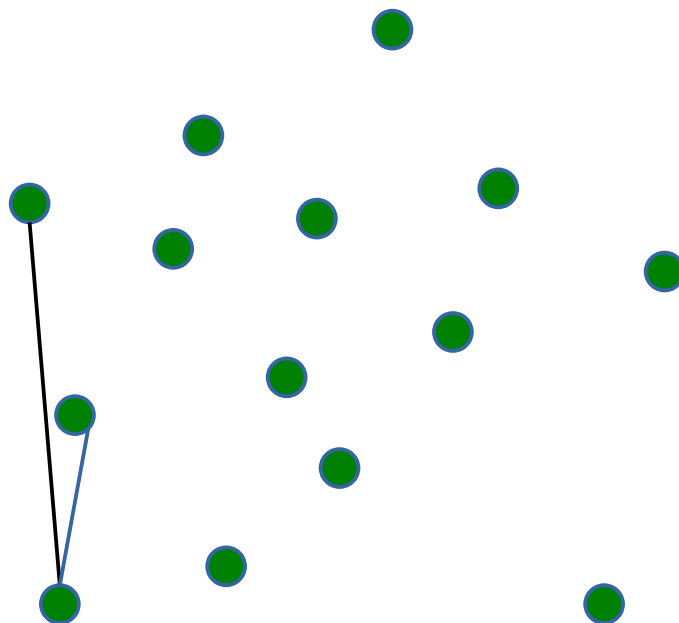




# Convex Hull

(envoltória convexa / fecho convexo)

---

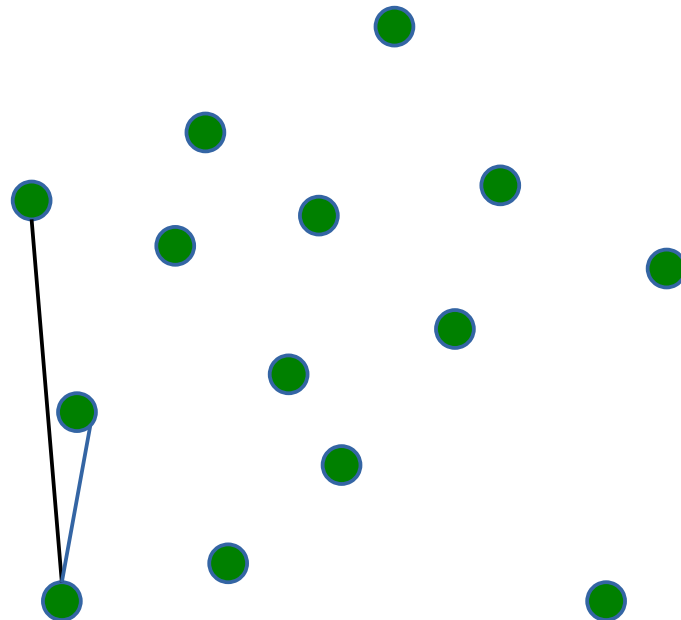


# Convex Hull

(envoltória convexa / fecho convexo)

---

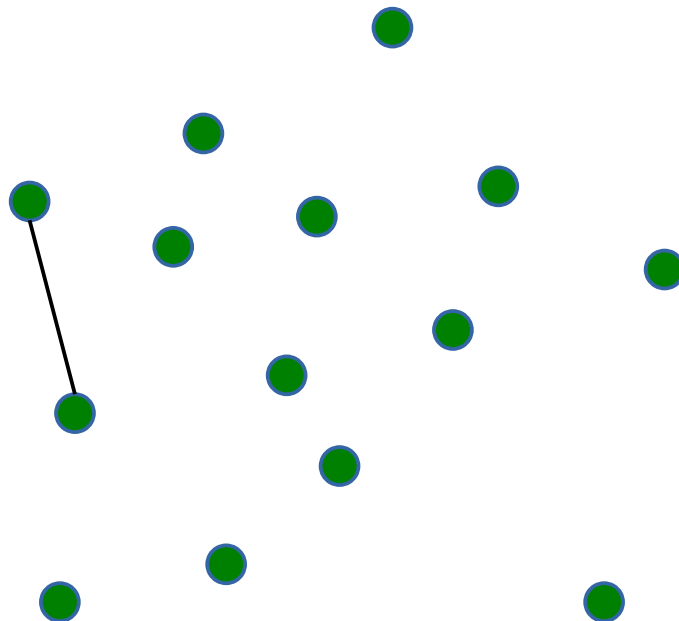
Virada à esquerda!



# Convex Hull

(envoltória convexa / fecho convexo)

---

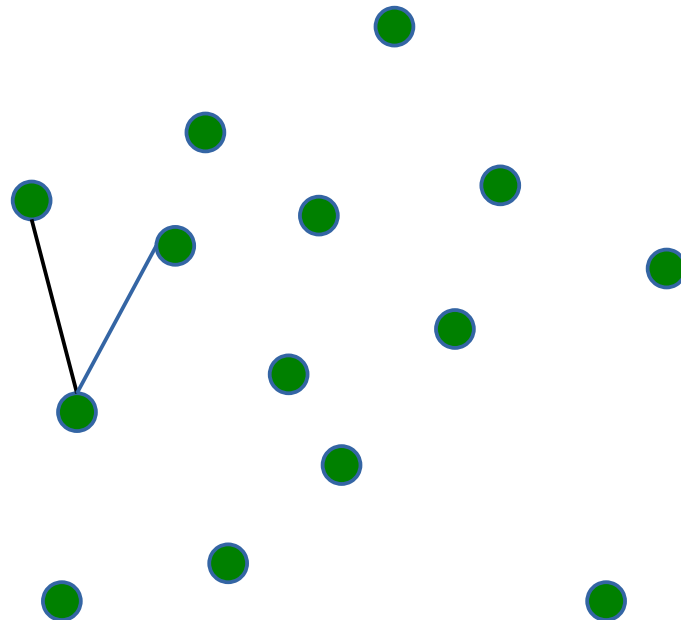


# Convex Hull

(envoltória convexa / fecho convexo)

---

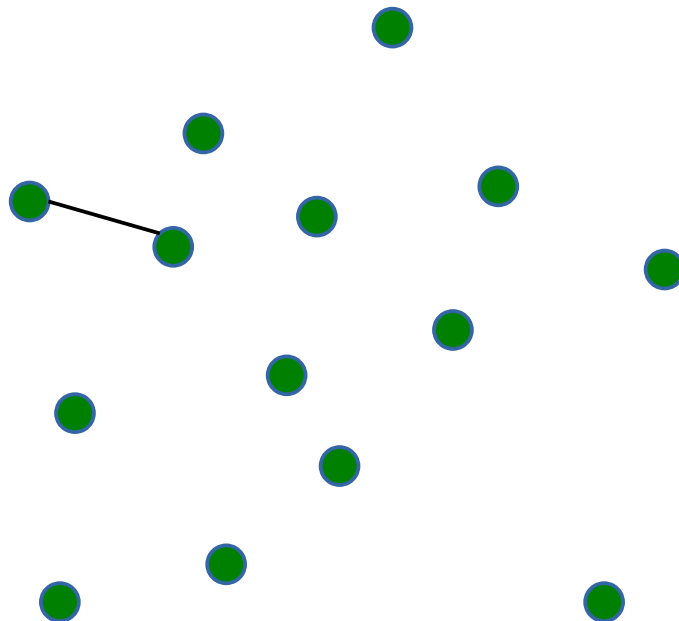
Virada à esquerda!



# Convex Hull

(envoltória convexa / fecho convexo)

---

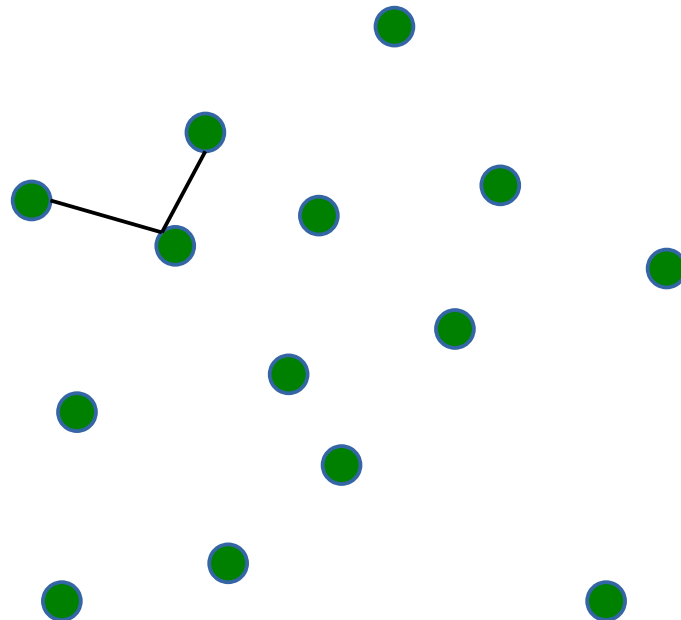


# Convex Hull

(envoltória convexa / fecho convexo)

---

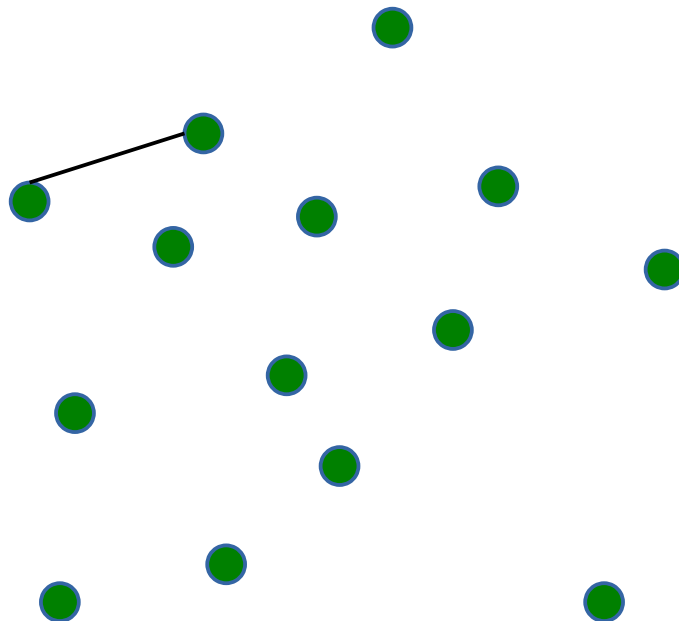
Virada à esquerda!



# Convex Hull

(envoltória convexa / fecho convexo)

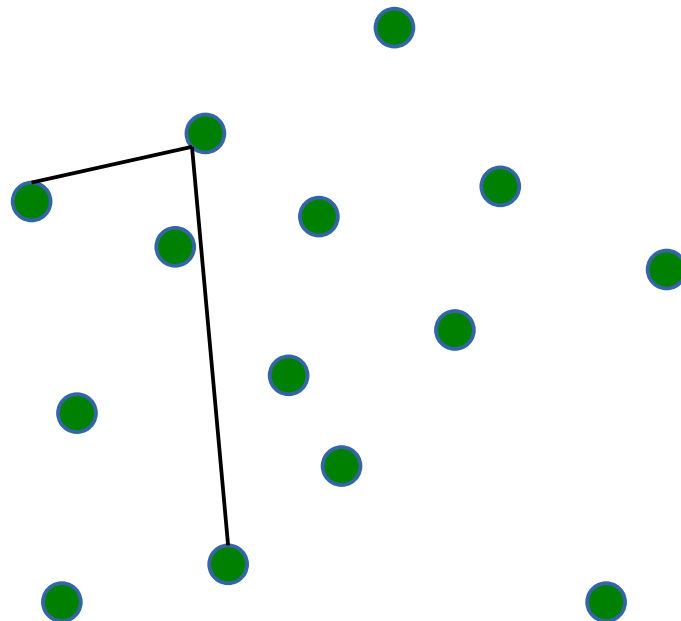
---



# Convex Hull

(envoltória convexa / fecho convexo)

---



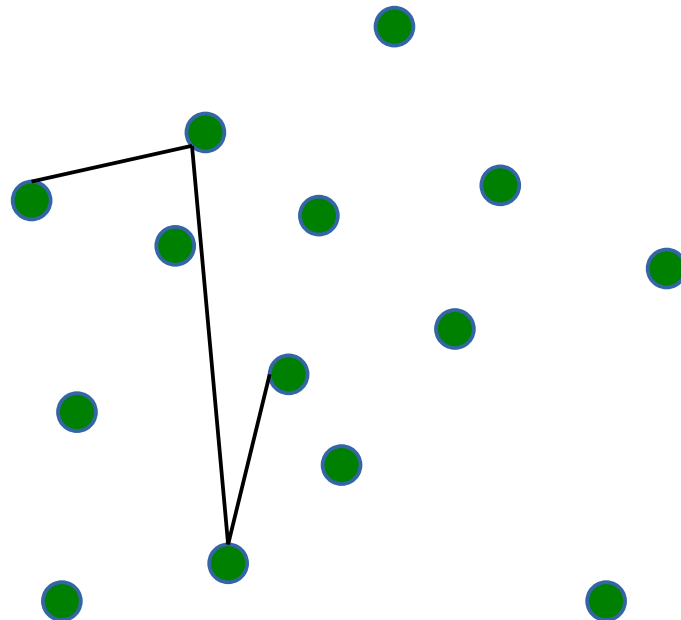


# Convex Hull

(envoltória convexa / fecho convexo)

---

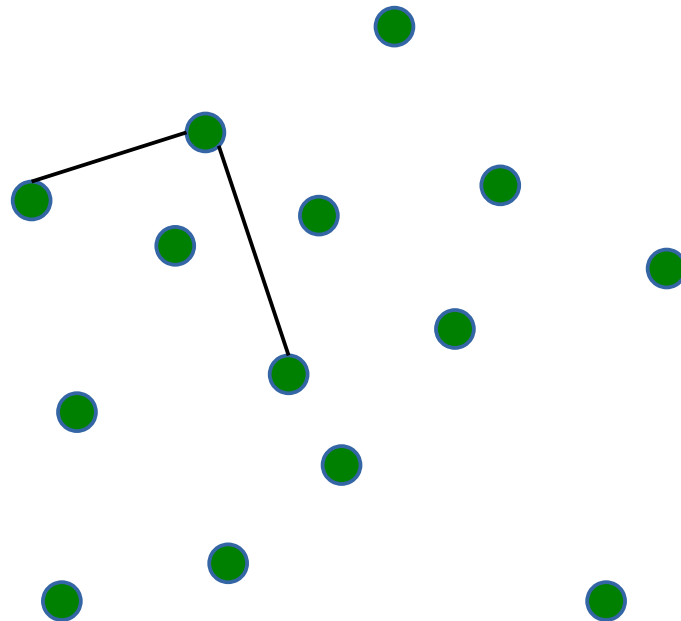
Virada à esquerda!



# Convex Hull

(envoltória convexa / fecho convexo)

---

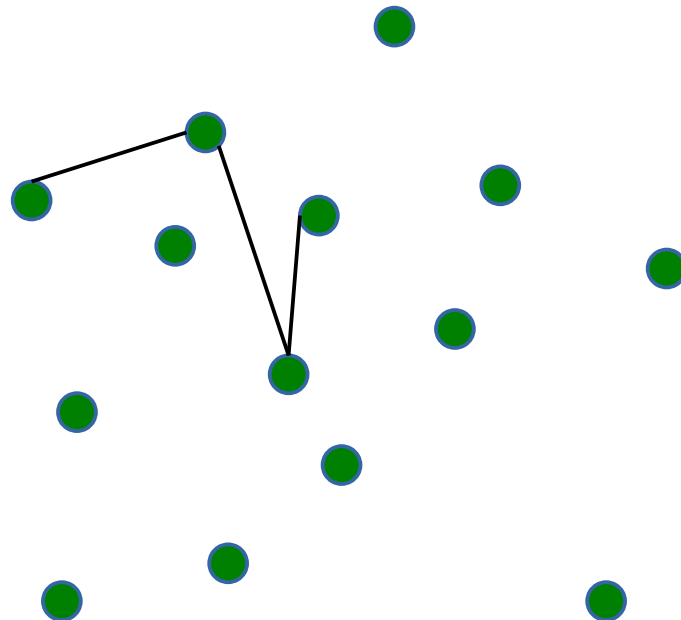


# Convex Hull

(envoltória convexa / fecho convexo)

---

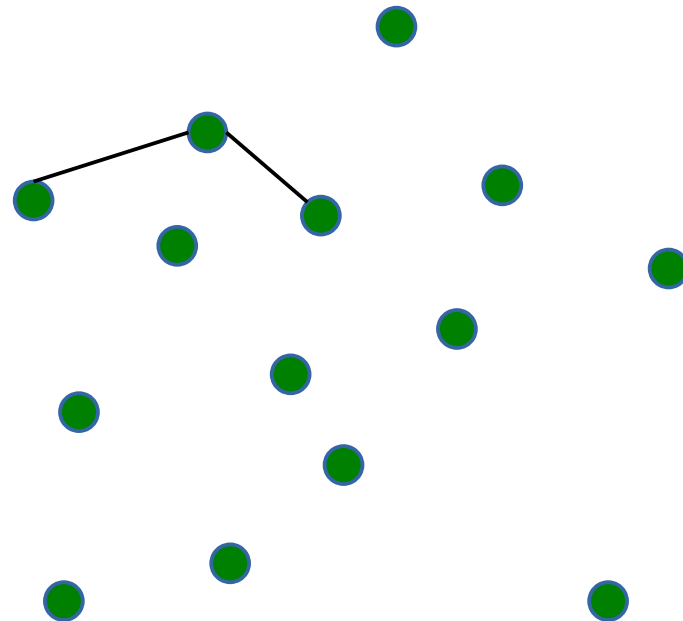
Virada à esquerda!



# Convex Hull

(envoltória convexa / fecho convexo)

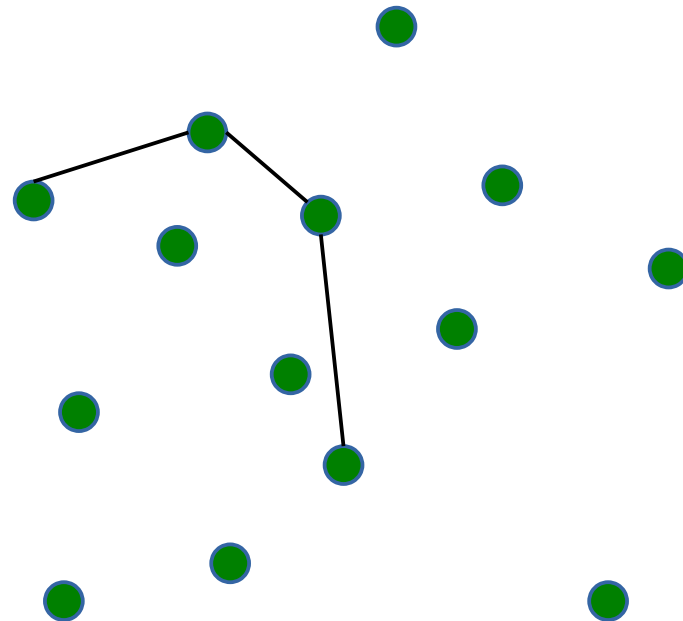
---



# Convex Hull

(envoltória convexa / fecho convexo)

---

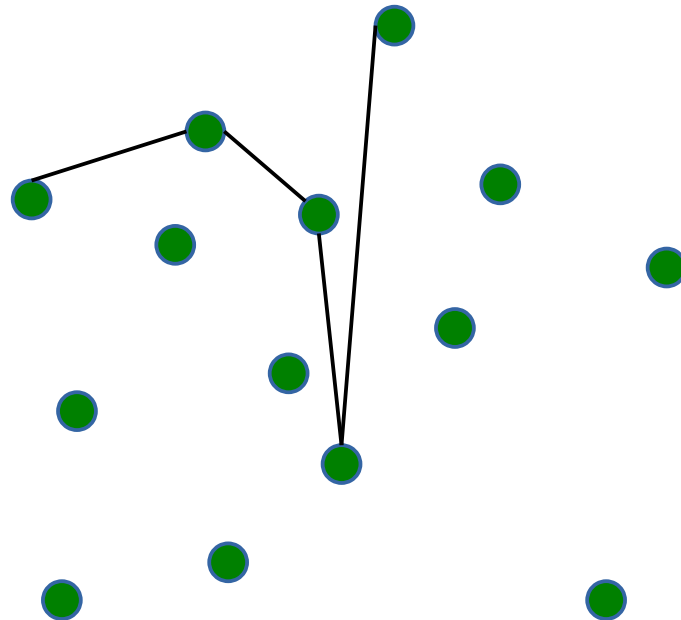


# Convex Hull

(envoltória convexa / fecho convexo)

---

Virada à esquerda!

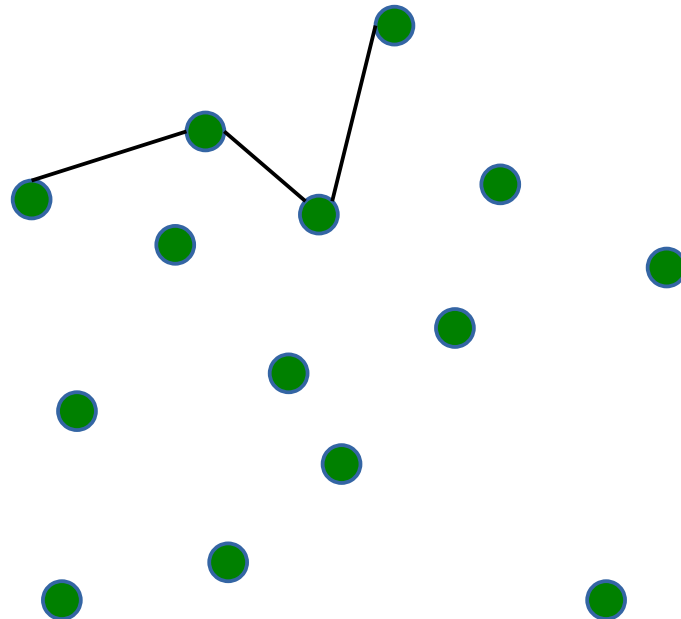


# Convex Hull

(envoltória convexa / fecho convexo)

---

Virada à esquerda!

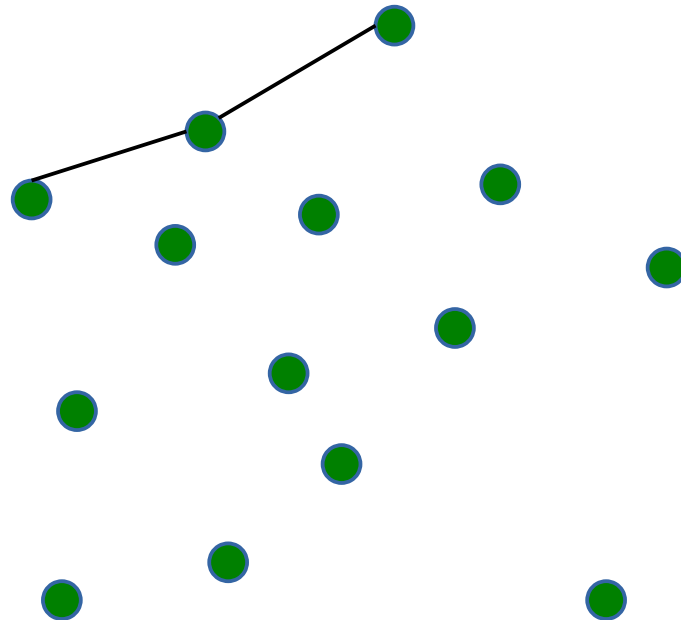


# Convex Hull

(envoltória convexa / fecho convexo)

---

Virada à esquerda!

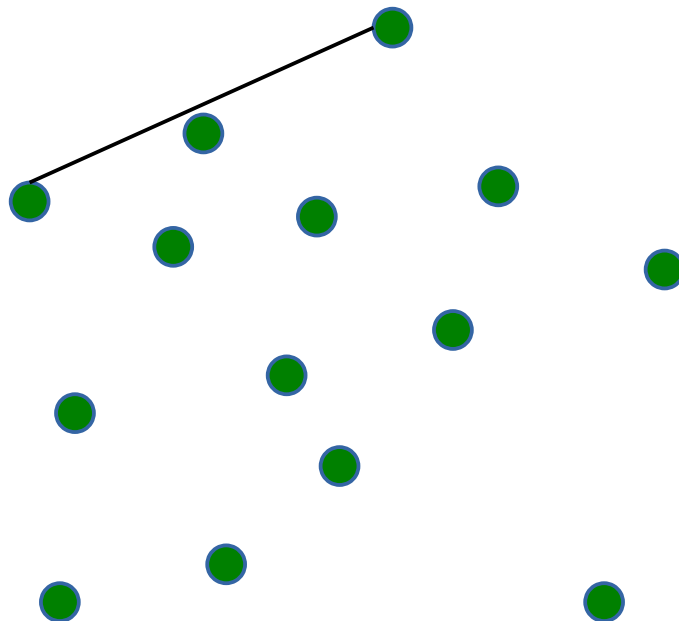




# Convex Hull

(envoltória convexa / fecho convexo)

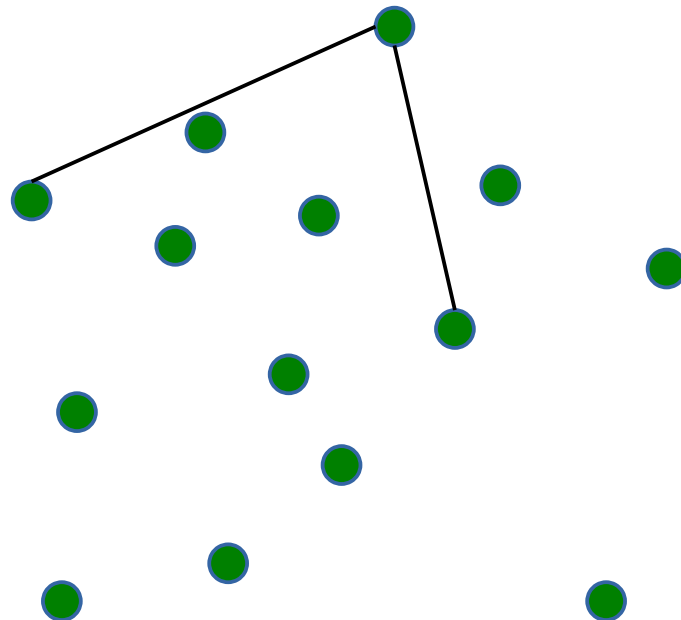
---



# Convex Hull

(envoltória convexa / fecho convexo)

---

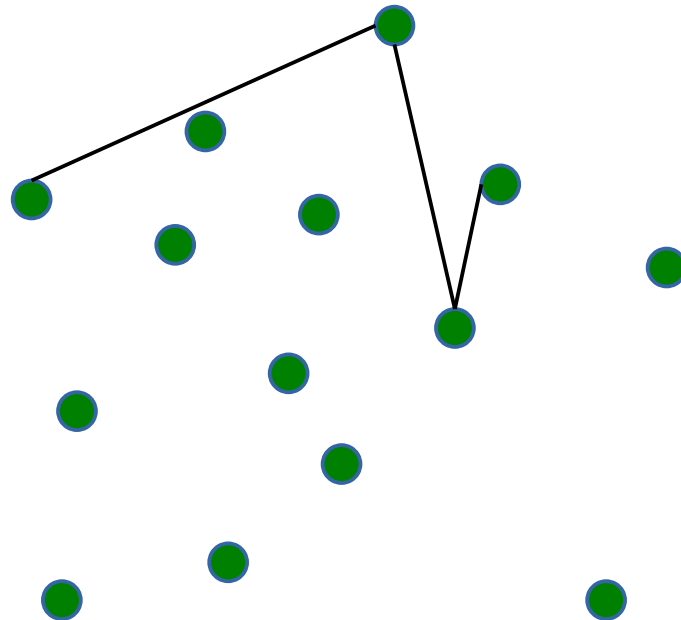


# Convex Hull

(envoltória convexa / fecho convexo)

---

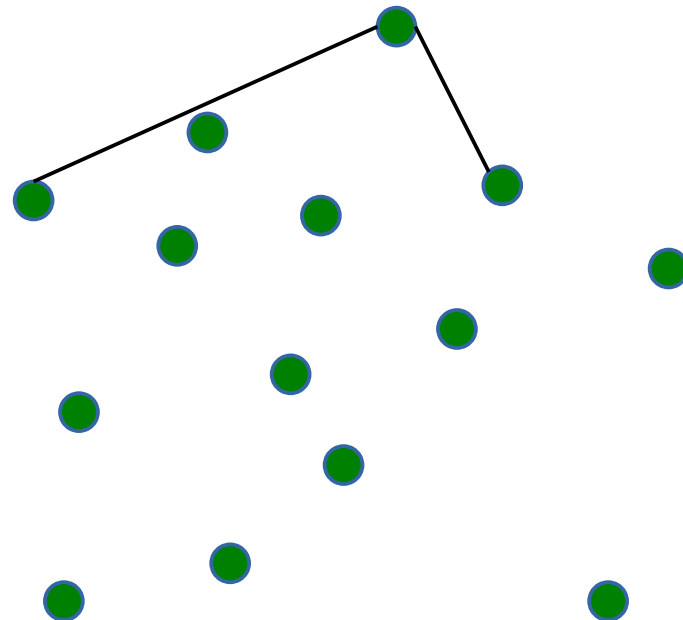
Virada à esquerda!



# Convex Hull

(envoltória convexa / fecho convexo)

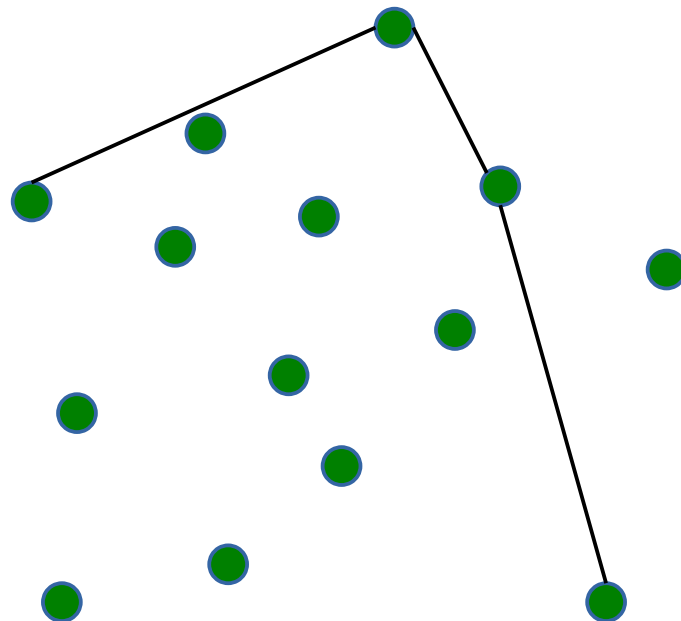
---



# Convex Hull

(envoltória convexa / fecho convexo)

---

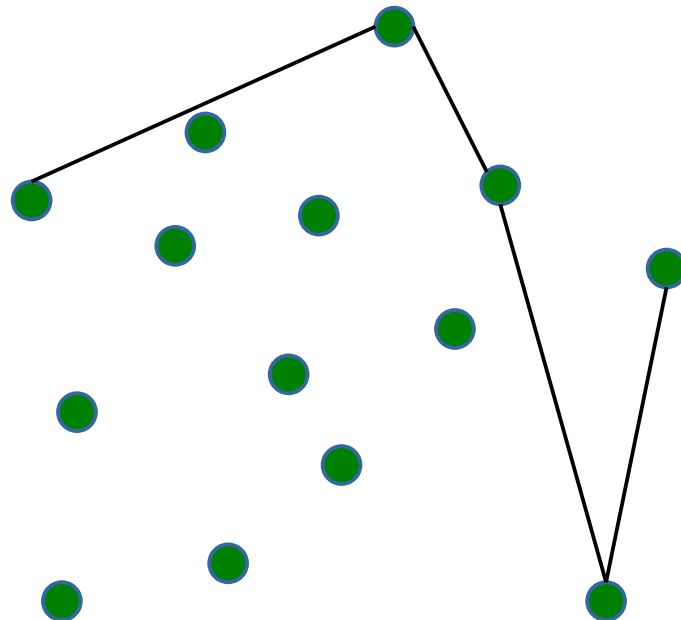


# Convex Hull

(envoltória convexa / fecho convexo)

---

Virada à esquerda!

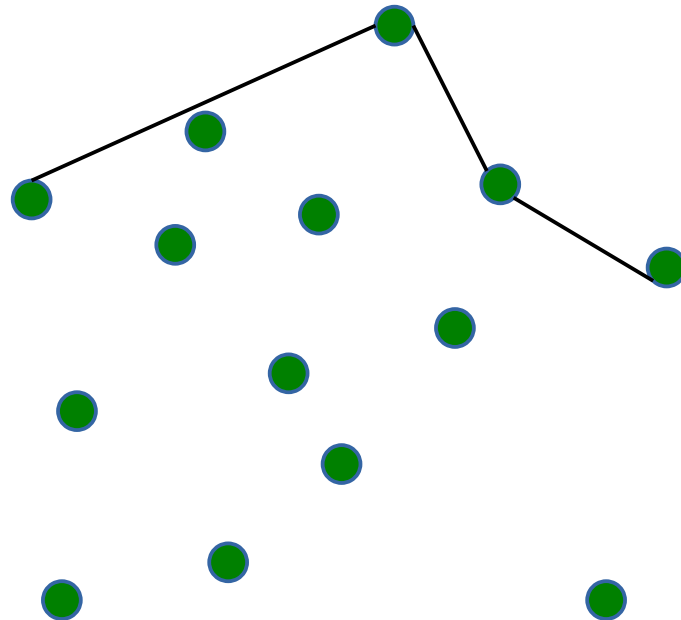


# Convex Hull

(envoltória convexa / fecho convexo)

---

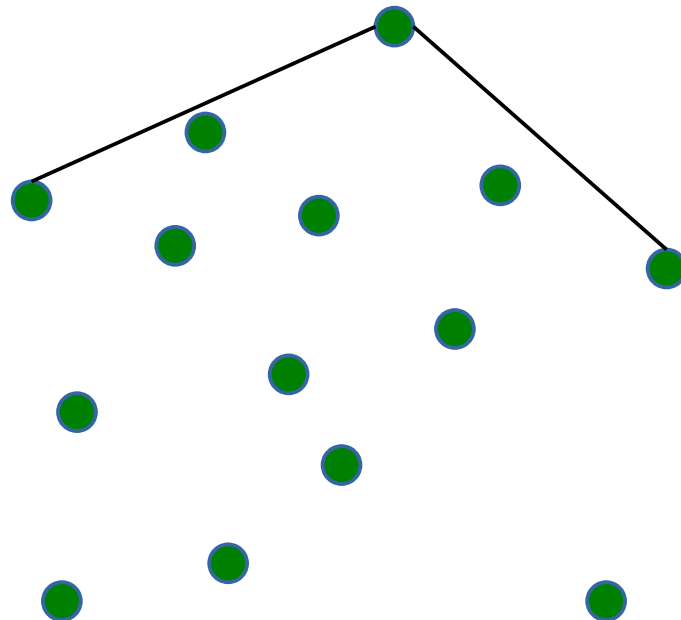
Virada à esquerda!



# Convex Hull

(envoltória convexa / fecho convexo)

---

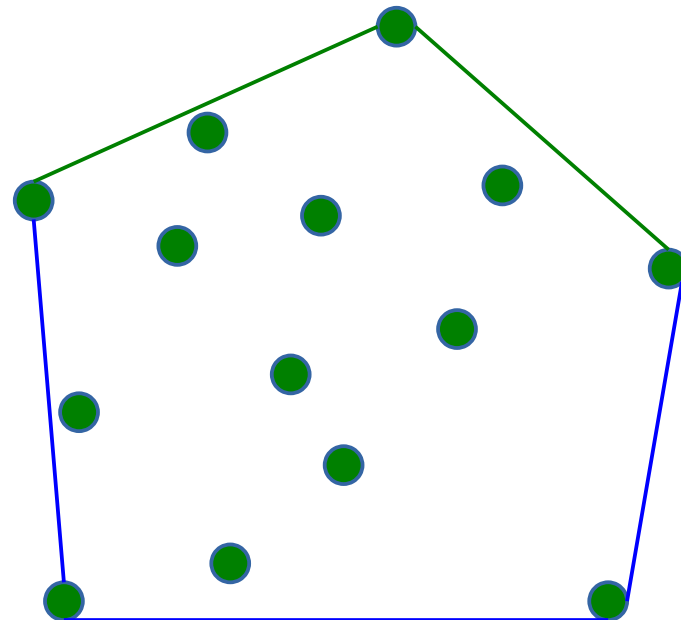




# Convex Hull

(envoltória convexa / fecho convexo)

---

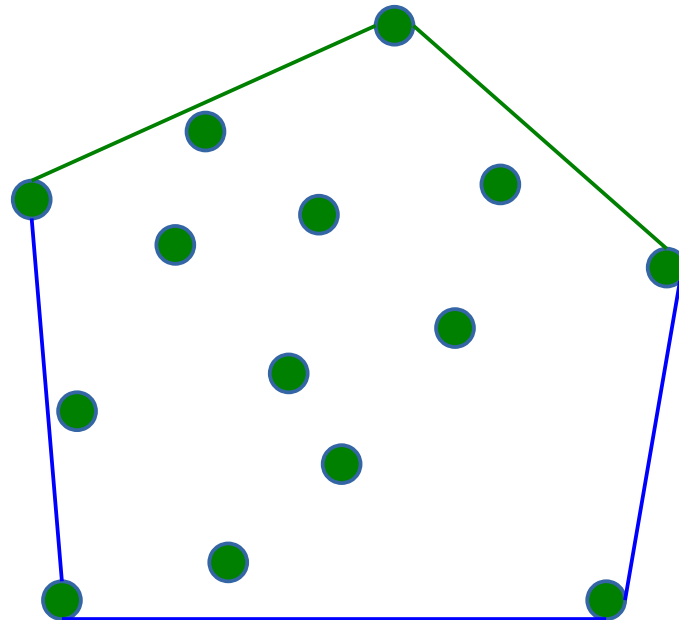


Idem para a parte inferior!

# Convex Hull

(envoltória convexa / fecho convexo)

---



Idem para a parte inferior!

Complexidade:  $n \log(n) + 2n = n \log(n)$

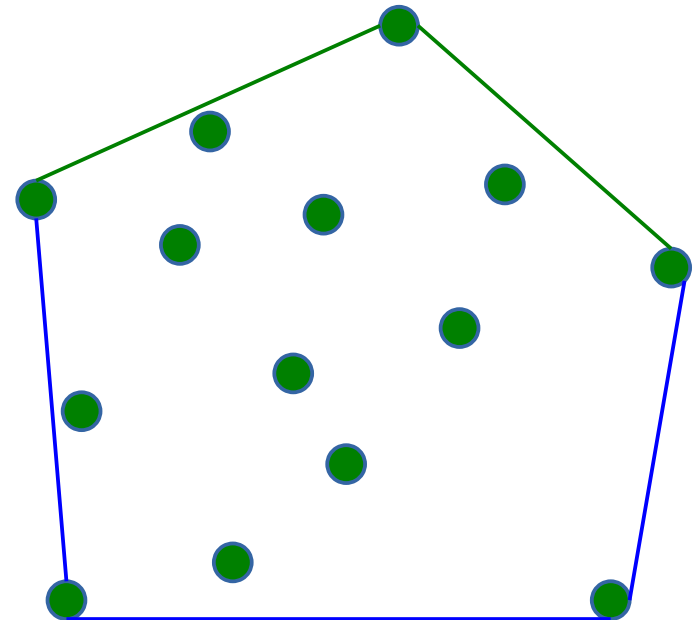
# Convex Hull

(envoltória convexa / fecho convexo)

---

Ex.: computador com 120 TOPS é capaz de realizar 10.000.000.000 de comparações se  $P$  está esq/dir de segmento de reta em um segundo.

Se tivermos uma nuvem com 1.000.000 de pontos, quanto tempo precisaríamos para Calcular o *convex hull*?



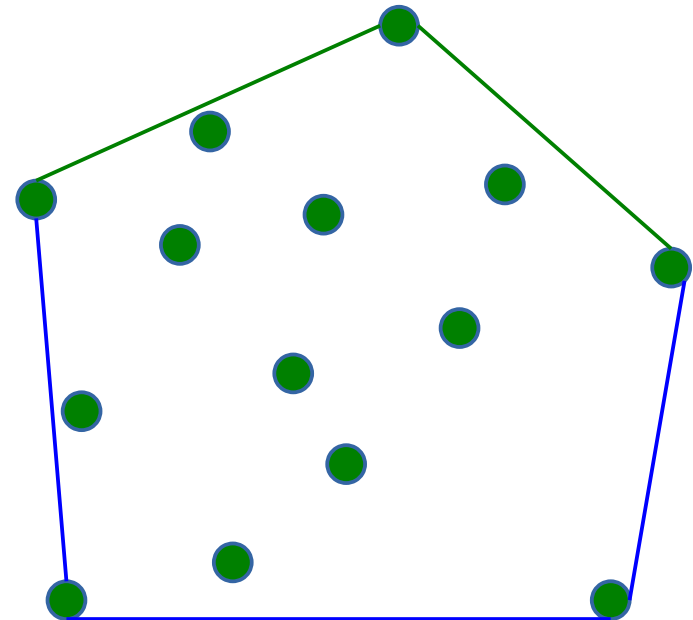
# Convex Hull

(envoltória convexa / fecho convexo)

---

Ex.: computador com 120 TOPS é capaz de realizar 10.000.000.000 de comparações se  $P$  está esq/dir de segmento de reta em um segundo.

Se tivermos uma nuvem com 1.000.000 de pontos, quanto tempo precisaríamos para Calcular o *convex hull*?



$$(*) \log_2(1.000.000) = \sim 19,93$$

Resposta: 0,002 segundos no  $n \log(n)$  e ...

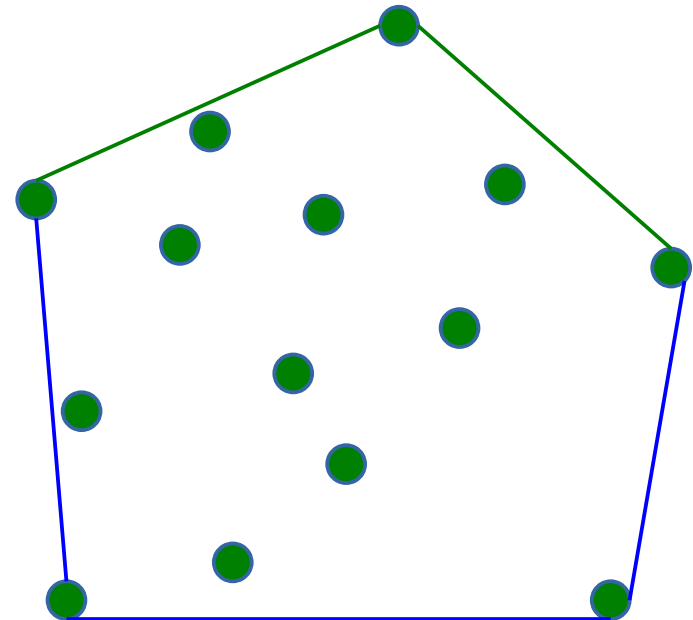
# Convex Hull

(envoltória convexa / fecho convexo)

---

Ex.: computador com 120 TOPS é capaz de realizar 10.000.000.000 de comparações se  $P$  está esq/dir de segmento de reta em um segundo.

Se tivermos uma nuvem com 1.000.000 de pontos, quanto tempo precisaríamos para Calcular o *convex hull*?



Resposta: 0,002 segundos no  $n \log(n)$  e 190,26 anos no algoritmo  $n^3$ .

# Jogos – detecção de colisão

---

Precisamos agora ler um ambiente formado por milhares de polígonos e precisamos exibí-los em uma alta taxa de quadros (mais de 60 FPS, por exemplo).

Mas não basta exibir o ambiente. Precisamos popular ele com NPCs. E esses NPCs não podem atravessar por dentro dos demais. Precisamos detectar e evitar colisão entre os NPCs.

Em vez de testarmos a colisão entre cada um dos polígonos, podemos verificar se existe alguma intersecção entre dois **polígonos convexos**.

# Considerações Finais

---

Aqui não estamos preocupados em estudar metodologias de programação ágil como Scrum, Kanban, Extreme Programming (XP), etc. O objetivo da disciplina é a construção de programas EFICIENTES.

Quando utilizamos uma linguagem de programação, é importante saber o funcionamento da mesma para tirar o máximo proveito dela evitando fazer códigos muito lentos. Para isto, precisamos conhecer quais estruturas de dados e otimizações podemos fazer nos nossos programas.